# Notion RAG Chat Agent

완전 통합 최종 개발 가이드 (v1.0)

---

📌 **문서 개요**
본 문서는 비개발자도 처음부터 끝까지 따라하여 **Notion 데이터를 기반으로 답변하는 사내 AI 챗봇**을 구축할 수 있도록 작성되었습니다.
모든 소스코드는 복사/붙여넣기만 하면 동작하도록 완전하게 제공됩니다.

### 목차 (Table of Contents)


- PART 0: 시작하기 전에 (개념 및 아키텍처)
- PART 1: 환경 설정 및 프로젝트 구조
- PART 2: 외부 API 설정 (Notion, Google, Jandi)
- PART 3: Docker Compose 설정
- PART 4: 백엔드 구현 (FastAPI + LangChain)
- PART 5: 프론트엔드 구현 (Streamlit)
- PART 6: 실행 및 사용 가이드
- PART 7: 아키텍처 다이어그램 상세

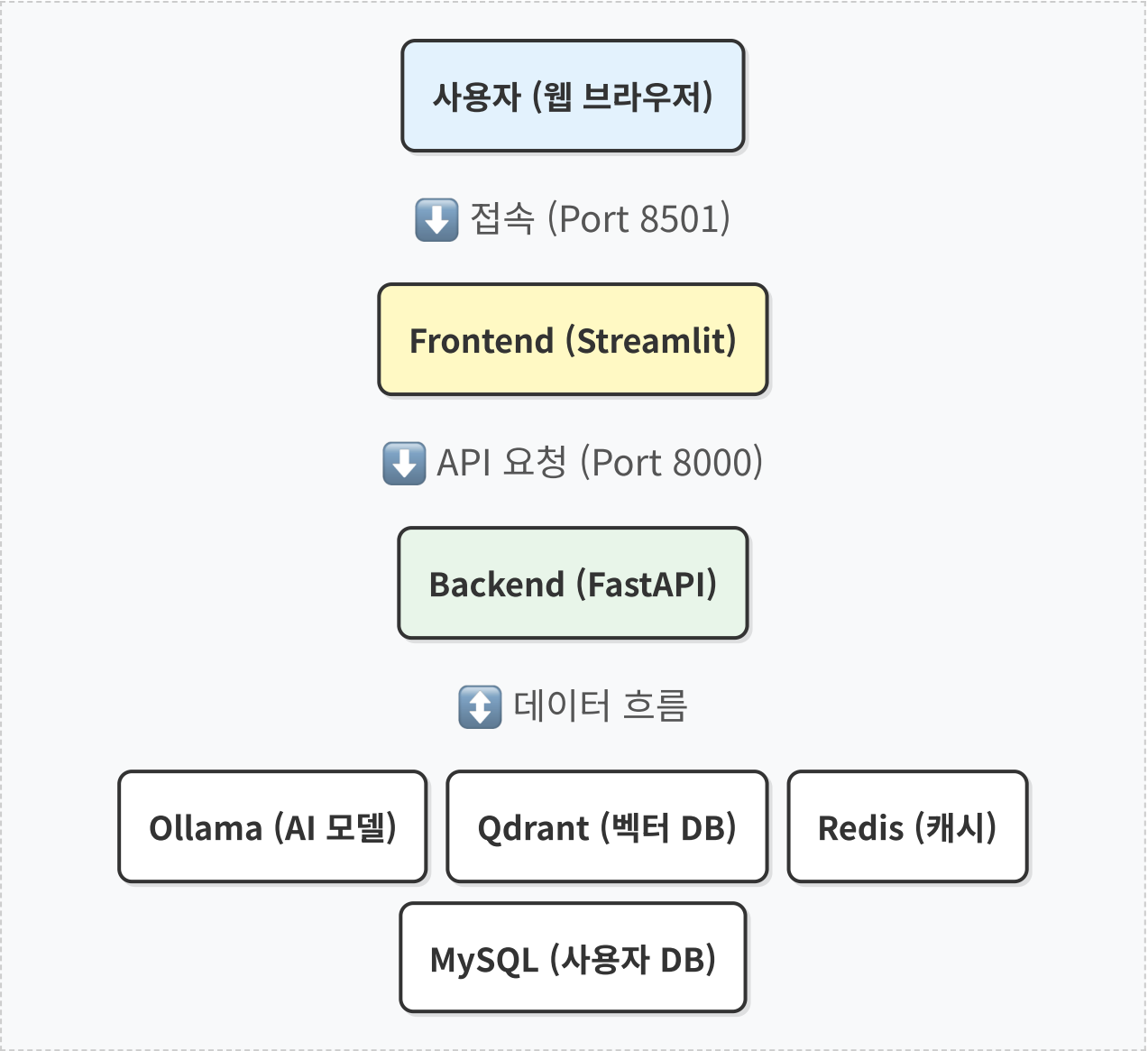

## PART 0: 시작하기 전에

### 1. RAG(검색 증강 생성)란 무엇인가?

AI 챗봇(LLM)은 기본적으로 학습하지 않은 여러분 회사의 비공개 정보를 모릅니다. **RAG(Retrieval-Augmented Generation)**는 이를 해결하기 위해 마치 **"오픈북 시험"**을 보는 것과 같습니다.

- **기존 챗봇:** 머릿속에 있는 지식만으로 답변 (회사 정보 모름)
- **RAG 챗봇:** 질문을 받으면 1) 회사 문서를 찾아보고(검색), 2) 찾은 내용을 참고해서 답변(생성)

### 2. 전체 시스템 아키텍처




## PART 1: 환경 설정 및 프로젝트 구조

### 1. 사전 준비물

- **Docker Desktop 설치:** Docker 공식 사이트에서 다운로드 후 설치 및 실행.
- **Git 설치 (선택):** 소스 코드 관리를 위해 권장.
- **텍스트 에디터:** VS Code (Visual Studio Code) 추천.

### 2. 프로젝트 폴더 구조 생성

바탕화면이나 원하는 위치에 `notion-rag-agent` 라는 폴더를 만들고, 아래 구조대로 하위 폴더와 빈 파일들을 미리 생성하세요.

```
      notion-rag-agent/ ├── docker-compose.yml ├── .env ├── backend/ │ ├── Dockerfile │ ├── requirements.txt │ ├── main.py │ ├── config.py │ ├── alembic.ini │ ├── alembic/ │ │ ├── env.py │ │ └── versions/ (빈 폴더) │ ├── core/ │ │ ├── rag_chain.py │ │ ├── memory.py │ │ ├── vector_store.py │ │ └── scheduler.py │ ├── services/ │ │ ├── notion_service.py │ │ ├── file_parser.py │ │ ├── jandi_service.py │ │ └── auth_service.py │ ├── models/ │ │ ├── database.py │ │ └── user.py │ └── api/ │ └── routes/ │ ├── chat.py │ ├── ingest.py │ ├── auth.py │ └── webhook.py ├── frontend/ │ ├── Dockerfile │ ├── requirements.txt │ └── app.py └── scripts/ └── ingest_notion.py
```

## PART 2: 외부 API 설정

### 1. Notion API 설정

1. Notion My Integrations 접속 후 '새 API 통합 만들기' 클릭.
2. 이름을 입력(예: RagBot)하고 제출.
3. **[내부 통합 시크릿]** 키를 복사해둡니다. (이것이 `NOTION_API_KEY` )
4. 연동하려는 Notion 페이지로 이동 → 우측 상단 `...` 클릭 → `연결 추가` → 방금 만든 통합 앱 선택.
5. 페이지 URL에서 데이터베이스 ID 추출: `https://notion.so/myworkspace/{DATABASE_ID}?v=...` (32자리 문자열)

### 2. Google OAuth 2.0 설정

1. Google Cloud Console 접속 및 새 프로젝트 생성.
2. 메뉴 → 'API 및 서비스' → 'OAuth 동의 화면' → '외부' 선택 후 만들기.
3. 앱 이름, 이메일 등 필수 정보 입력 후 저장.
4. '사용자 인증 정보' → '사용자 인증 정보 만들기' → 'OAuth 클라이언트 ID'.
5. 애플리케이션 유형: **웹 애플리케이션**.
6. 승인된 리디렉션 URI: `http://localhost:8000/auth/callback` 추가.
7. 생성된 **클라이언트 ID**와 **클라이언트 보안 비밀(Secret)** 복사.

### 3. .env 파일 작성

프로젝트 최상위 폴더의 `.env` 파일에 아래 내용을 채워 넣으세요.

.env

```
# --- Notion 설정 ---
NOTION_API_KEY=secret_여기에_노션_키_붙여넣기
NOTION_DATABASE_ID=여기에_데이터베이스_ID_붙여넣기

# --- Google OAuth 설정 ---
GOOGLE_CLIENT_ID=여기에_구글_클라이언트_ID
GOOGLE_CLIENT_SECRET=여기에_구글_시크릿_키
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/callback

# --- 보안 (JWT & 초대코드) ---
JWT_SECRET_KEY=super_secret_random_key_change_this
JWT_ALGORITHM=HS256
INVITE_CODE=welcome1234

# --- 데이터베이스 (MySQL) ---
MYSQL_ROOT_PASSWORD=rootpassword
MYSQL_PASSWORD=userpassword
MYSQL_DATABASE=notion_rag
MYSQL_USER=raguser

# --- 잔디(Jandi) 웹훅 ---
JANDI_WEBHOOK_URL=https://wh.jandi.com/connect-api/webhook/27706920/fab6a04
ec1f015cc276e111ffe8ac5a6

# --- 기타 설정 ---
SYNC_INTERVAL_HOURS=1
OLLAMA_BASE_URL=http://ollama:11434
```

## PART 3: Docker Compose 설정

컨테이너 오케스트레이션을 위한 설정입니다. `docker-compose.yml` 파일에 작성하세요.

docker-compose.yml

```yaml
version: '3.8'

services:
  # 1. AI Model Service
  ollama:
    image: ollama/ollama:latest
    container_name: notion-rag-ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    restart: always

  # 2. Vector Database
  qdrant:
    image: qdrant/qdrant:latest
    container_name: notion-rag-qdrant
    ports:
      - "6333:6333"
    volumes:
      - qdrant_data:/qdrant/storage
    restart: always

  # 3. Cache & Session Store
  redis:
    image: redis:7-alpine
    container_name: notion-rag-redis
    ports:
      - "6379:6379"
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    restart: always

  # 4. User Database (MySQL)
  mysql:
    image: mysql:8.0
    container_name: notion-rag-mysql
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      retries: 5
    restart: always

  # 5. Backend Application
  backend:
    build: ./backend
    container_name: notion-rag-backend
    ports:
      - "8000:8000"
    env_file: .env
    depends_on:
```

```
      mysql:
        condition: service_healthy
      qdrant:
        condition: service_started
      redis:
        condition: service_started
      ollama:
        condition: service_started
    volumes:
      - ./backend:/app
    restart: always

  # 6. Frontend Application
  frontend:
    build: ./frontend
    container_name: notion-rag-frontend
    ports:
      - "8501:8501"
    env_file: .env
    depends_on:
      - backend
    volumes:
      - ./frontend:/app
    restart: always

volumes:
  ollama_data:
  qdrant_data:
  redis_data:
  mysql_data:

networks:
  default:
    name: rag-network
```

## PART 4: 백엔드 구현 (FastAPI)

### 1. 환경 설정 및 의존성

backend/requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.32.0
langchain==0.3.7
langchain-community==0.3.7
langchain-ollama==0.2.0
qdrant-client==1.12.0
redis==5.2.0
sqlalchemy==2.0.36
aiomysql==0.2.0
pymysql==1.1.1
alembic==1.13.3
python-jose[cryptography]==3.3.0
httpx==0.27.2
pydantic-settings==2.5.2
```

```
apscheduler==3.10.4
notion-client==2.2.1
PyMuPDF==1.24.13
python-docx==1.1.2
Pillow==11.0.0
pytesseract==0.3.13
python-multipart==0.0.12
aiofiles==24.1.0
```

backend/Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

# 시스템 의존성 설치 (OCR용 tesseract 포함)
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    libtesseract-dev \
    gcc \
    default-libmysqlclient-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reloa
d"]
```

backend/config.py

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    # Notion 설정
    NOTION_API_KEY: str
    NOTION_DATABASE_ID: str = ""

    # Ollama 설정
    OLLAMA_BASE_URL: str = "http://ollama:11434"
    OLLAMA_MODEL: str = "llama3.2:3b"
    OLLAMA_EMBED_MODEL: str = "nomic-embed-text"

    # Qdrant 설정
    QDRANT_HOST: str = "qdrant"
    QDRANT_PORT: int = 6333
    QDRANT_COLLECTION: str = "notion_docs"

    # Redis 설정
    REDIS_HOST: str = "redis"
    REDIS_PORT: int = 6379
    REDIS_DB: int = 0

    # MySQL 설정
    MYSQL_HOST: str = "mysql"
```

```
    MYSQL_PORT: int = 3306
    MYSQL_USER: str = "raguser"
    MYSQL_PASSWORD: str
    MYSQL_DATABASE: str = "notion_rag"

    # Google OAuth 설정
    GOOGLE_CLIENT_ID: str
    GOOGLE_CLIENT_SECRET: str
    GOOGLE_REDIRECT_URI: str = "http://localhost:8000/auth/callback"

    # JWT 설정
    JWT_SECRET_KEY: str
    JWT_ALGORITHM: str = "HS256"
    JWT_EXPIRE_HOURS: int = 24

    # 초대코드 설정
    INVITE_CODE: str

    # 잔디 웹훅
    JANDI_WEBHOOK_URL: str = ""

    # 동기화 설정
    SYNC_INTERVAL_HOURS: int = 1

    class Config:
        env_file = ".env"
        extra = "ignore" # 정의되지 않은 환경변수 무시

@lru_cache()
def get_settings():
    return Settings()
```

## 2. 데이터베이스 모델 및 설정

backend/models/database.py

```
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker,
AsyncSession
from sqlalchemy.orm import DeclarativeBase
from typing import AsyncGenerator
from config import get_settings

settings = get_settings()

# 비동기 엔진 생성 URL 구성
DATABASE_URL = f"mysql+aiomysql://{settings.MYSQL_USER}:{settings.MYSQL_PAS
SWORD}@{settings.MYSQL_HOST}:{settings.MYSQL_PORT}/{settings.MYSQL_DATABAS
E}"

# 엔진 생성
engine = create_async_engine(
    DATABASE_URL,
    echo=True, # 쿼리 로그 출력
    pool_pre_ping=True, # 연결 끊김 방지
    pool_recycle=3600,
)
```

```
# 세션 팩토리
async_session_maker = async_sessionmaker(engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

# 의존성 주입용 DB 세션 제너레이터
async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_maker() as session:
        try:
            yield session
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()
```

backend/models/user.py

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Boolean, DateTime, func
from datetime import datetime
from .database import Base

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=F
alse, index=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    google_id: Mapped[str] = mapped_column(String(255), unique=True, nullab
le=False)
    picture_url: Mapped[str | None] = mapped_column(String(500), nullable=T
rue)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=f
unc.now())
    last_login_at: Mapped[datetime | None] = mapped_column(DateTime, onupda
te=func.now(), nullable=True)
```

## 3. Alembic 마이그레이션

`backend/alembic/env.py` 파일과 마이그레이션 스크립트를 작성합니다.

backend/alembic/env.py

```
import asyncio
from logging.config import fileConfig
from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import async_engine_from_config
from alembic import context
import os
import sys
```

```
# 프로젝트 루트 경로 추가
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file_
_))))

from models.database import Base, DATABASE_URL
from models.user import User # 모델 임포트 필수

config = context.config

if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata

# 동기 드라이버 URL로 변환 (Alembic은 내부적으로 동기 처리 필요 가능성 대비)
# 하지만 여기서는 AsyncEngine을 사용하여 비동기 마이그레이션 실행
config.set_main_option("sqlalchemy.url", DATABASE_URL)

def run_migrations_offline() -> None:
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online() -> None:
    connectable = async_engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

    await connectable.dispose()

def do_run_migrations(connection: Connection) -> None:
    context.configure(connection=connection, target_metadata=target_metadat
a)
    with context.begin_transaction():
        context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())
```

backend/alembic/versions/0001_create_users_table.py

```
"""create users table

Revision ID: 0001
Revises:
Create Date: 2024-01-01 00:00:00.000000
```

```
"""
from typing import Sequence, Union
from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = '0001'
down_revision: Union[str, None] = None
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    # Users 테이블 생성
    op.create_table('users',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('email', sa.String(length=255), nullable=False),
    sa.Column('name', sa.String(length=255), nullable=False),
    sa.Column('google_id', sa.String(length=255), nullable=False),
    sa.Column('picture_url', sa.String(length=500), nullable=True),
    sa.Column('is_active', sa.Boolean(), nullable=False),
    sa.Column('created_at', sa.DateTime(), server_default=sa.text('now()'),
nullable=False),
    sa.Column('last_login_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('email'),
    sa.UniqueConstraint('google_id')
    )
    op.create_index(op.f('ix_users_email'), 'users', ['email'], unique=Tru
e)

def downgrade() -> None:
    op.drop_index(op.f('ix_users_email'), table_name='users')
    op.drop_table('users')
```

backend/alembic.ini

```
# alembic.ini (기본 설정)
[alembic]
script_location = alembic
prepend_sys_path = .
version_path_separator = os
sqlalchemy.url = driver://user:pass@localhost/dbname # env.py에서 덮어씌워짐

[post_write_hooks]

[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARN
handlers = console
qualname =
```

```
[logger_sqlalchemy]
level = WARN
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

## 4. 핵심 로직 (Core & Services)

backend/core/vector_store.py

```
from qdrant_client import QdrantClient
from qdrant_client.http import models
from langchain_ollama import OllamaEmbeddings
from config import get_settings

settings = get_settings()

class VectorStore:
    def __init__(self):
        self.client = QdrantClient(host=settings.QDRANT_HOST, port=setting
s.QDRANT_PORT)
        self.embeddings = OllamaEmbeddings(
            base_url=settings.OLLAMA_BASE_URL,
            model=settings.OLLAMA_EMBED_MODEL
        )
        self.collection_name = settings.QDRANT_COLLECTION
        self._ensure_collection()

    def _ensure_collection(self):
        """컬렉션이 없으면 생성"""
        collections = self.client.get_collections().collections
        if not any(c.name == self.collection_name for c in collections):
            self.client.create_collection(
                collection_name=self.collection_name,
                vectors_config=models.VectorParams(size=768, distance=model
s.Distance.COSINE)
            )

    def add_texts(self, texts: list[str], metadatas: list[dict]):
        """텍스트 임베딩 후 저장"""
        embeddings = self.embeddings.embed_documents(texts)
        points = [
            models.PointStruct(
                id=idx,  # 실제 구현시 UUID 사용 권장
```

```
                vector=embedding,
                payload={"text": text, **metadata}
            )
            for idx, (text, embedding, metadata) in enumerate(zip(texts, em
beddings, metadatas))
        ]
        # Qdrant에 업로드 (배치 처리는 생략, 간단 구현)
        # 실제로는 고유 ID 관리 필요. 여기서는 간단히 덮어쓰기 로직이 필요할 수 있음.
        # LangChain Qdrant wrapper를 쓰는 것이 편하지만, 구조 이해를 위해 직접 구
현하거나 LangChain 연결
        pass

    def get_langchain_store(self):
        """LangChain용 VectorStore 객체 반환"""
        from langchain_community.vectorstores import Qdrant
        return Qdrant(
            client=self.client,
            collection_name=self.collection_name,
            embeddings=self.embeddings
        )

vector_store = VectorStore()
```

backend/core/rag_chain.py

```
from langchain_ollama import ChatOllama
from langchain.prompts import ChatPromptTemplate
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from .vector_store import vector_store
from config import get_settings

settings = get_settings()

def get_rag_chain():
    # 1. LLM 설정
    llm = ChatOllama(
        base_url=settings.OLLAMA_BASE_URL,
        model=settings.OLLAMA_MODEL,
        temperature=0.1
    )

    # 2. Retriever 설정
    retriever = vector_store.get_langchain_store().as_retriever(
        search_kwargs={"k": 3} # 상위 3개 문서 검색
    )

    # 3. 프롬프트 템플릿
    prompt = ChatPromptTemplate.from_template("""
    아래의 문맥(Context)을 사용하여 질문에 답변해주세요.
    답변은 한국어로 작성하고, 모르는 내용은 "문서에 내용이 없습니다"라고 하세요.


    {context}


    질문: {input}
    """)
```

```
    # 4. 체인 연결
    document_chain = create_stuff_documents_chain(llm, prompt)
    retrieval_chain = create_retrieval_chain(retriever, document_chain)

    return retrieval_chain
```

backend/core/memory.py

```
import redis
import json
from config import get_settings

settings = get_settings()

class ChatMemory:
    def __init__(self):
        self.redis = redis.Redis(
            host=settings.REDIS_HOST,
            port=settings.REDIS_PORT,
            db=settings.REDIS_DB,
            decode_responses=True
        )

    def add_message(self, session_id: str, role: str, content: str):
        """메시지 저장 (List 사용)"""
        message = json.dumps({"role": role, "content": content})
        self.redis.rpush(f"chat:{session_id}", message)

    def get_history(self, session_id: str):
        """전체 히스토리 조회"""
        messages = self.redis.lrange(f"chat:{session_id}", 0, -1)
        return [json.loads(m) for m in messages]

    def delete_session(self, session_id: str):
        self.redis.delete(f"chat:{session_id}")

chat_memory = ChatMemory()
```

backend/services/auth_service.py

```
import httpx
from datetime import datetime, timedelta
from jose import jwt, JWTError
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from models.user import User
from config import get_settings

settings = get_settings()

async def get_google_user_info(code: str):
    """Google 인증 코드로 사용자 정보 조회"""
    async with httpx.AsyncClient() as client:
        # 1. 토큰 교환
        token_res = await client.post("https://oauth2.googleapis.com/toke
n", data={
            "code": code,
            "client_id": settings.GOOGLE_CLIENT_ID,
```

```
            "client_secret": settings.GOOGLE_CLIENT_SECRET,
            "redirect_uri": settings.GOOGLE_REDIRECT_URI,
            "grant_type": "authorization_code"
        })
        token_data = token_res.json()
        access_token = token_data.get("access_token")

        # 2. 사용자 정보 조회
        user_res = await client.get("https://www.googleapis.com/oauth2/v2/u
serinfo", headers={
            "Authorization": f"Bearer {access_token}"
        })
        return user_res.json()

def create_access_token(data: dict):
    """JWT 토큰 생성"""
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(hours=settings.JWT_EXPIRE_HOURS)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, settings.JWT_SECRET_KEY, algorithm=setting
s.JWT_ALGORITHM)

async def get_or_create_user(db: AsyncSession, google_info: dict) -> User:
    """사용자 조회 또는 생성"""
    result = await db.execute(select(User).where(User.google_id == google_i
nfo['id']))
    user = result.scalar_one_or_none()

    if not user:
        # 신규 사용자는 여기서 바로 생성하지 않고, 초대코드 확인 로직에서 처리할 수도
있음
        # 본 가이드에서는 자동 가입 처리
        user = User(
            email=google_info['email'],
            name=google_info['name'],
            google_id=google_info['id'],
            picture_url=google_info.get('picture', '')
        )
        db.add(user)
        await db.commit()
        await db.refresh(user)

    return user
```

backend/api/routes/auth.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from models.database import get_db
from services.auth_service import get_google_user_info, create_access_toke
n, get_or_create_user
from config import get_settings

router = APIRouter()
settings = get_settings()

@router.get("/auth/google")
def login_google():
    """구글 로그인 URL 반환"""
    scope = "https://www.googleapis.com/auth/userinfo.email https://www.goo
```

```
gleapis.com/auth/userinfo.profile"
    return {
        "url": f"https://accounts.google.com/o/oauth2/v2/auth?response_type
=code&client_id={settings.GOOGLE_CLIENT_ID}&redirect_uri={settings.GOOGLE_R
EDIRECT_URI}&scope={scope}&access_type=offline"
    }

@router.get("/auth/callback")
async def auth_callback(code: str, db: AsyncSession = Depends(get_db)):
    """구글 콜백 처리"""
    try:
        user_info = await get_google_user_info(code)
        user = await get_or_create_user(db, user_info)

        token = create_access_token({"sub": user.email, "user_id": user.i
d})
        return {"access_token": token, "token_type": "bearer", "user": use
r.email}
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
```

backend/api/routes/chat.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from core.rag_chain import get_rag_chain
from core.memory import chat_memory
import uuid

router = APIRouter()

class ChatRequest(BaseModel):
    message: str
    session_id: str = None

@router.post("/api/chat")
async def chat(request: ChatRequest):
    session_id = request.session_id or str(uuid.uuid4())

    # 1. RAG 체인 실행
    chain = get_rag_chain()
    response = await chain.ainvoke({"input": request.message})

    answer = response["answer"]
    sources = [doc.metadata.get("source", "Unknown") for doc in response.ge
t("context", [])]

    # 2. 메모리 저장
    chat_memory.add_message(session_id, "user", request.message)
    chat_memory.add_message(session_id, "assistant", answer)

    return {
        "session_id": session_id,
        "response": answer,
        "sources": list(set(sources)) # 중복 제거
    }

@router.get("/api/chats/{session_id}/history")
```

```
def get_history(session_id: str):
    return chat_memory.get_history(session_id)
```

backend/main.py

```
from fastapi import FastAPI
from api.routes import chat, auth
from config import get_settings
import uvicorn

app = FastAPI(title="Notion RAG Agent")
settings = get_settings()

# 라우터 등록
app.include_router(chat.router)
app.include_router(auth.router)

@app.get("/")
def health_check():
    return {"status": "ok", "service": "Notion RAG Agent"}

if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

## PART 5: 프론트엔드 구현 (Streamlit)

frontend/Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["streamlit", "run", "app.py", "--server.port", "8501", "--server.addre
ss", "0.0.0.0"]
```

frontend/app.py

```
import streamlit as st
import requests
import uuid

st.set_page_config(page_title="사내 AI 챗봇", layout="wide")

API_URL = "http://backend:8000"  # Docker 내부 네트워크 통신

# 세션 상태 초기화
if "session_id" not in st.session_state:
    st.session_state.session_id = str(uuid.uuid4())
if "messages" not in st.session_state:
    st.session_state.messages = []
if "token" not in st.session_state:
    st.session_state.token = None
```

```
def login():
    st.title("🔐 로그인")
    if st.button("Google 계정으로 로그인"):
        # 실제로는 백엔드에서 받은 URL로 리다이렉트 필요
        # Streamlit 특성상 팝업이나 링크 제공 방식으로 구현
        res = requests.get(f"{API_URL}/auth/google")
        if res.status_code == 200:
            url = res.json()["url"]
            st.markdown(f"[로그인 링크 클릭]({url})", unsafe_allow_html=True)
            st.info("로그인 후 발급받은 토큰을 아래 입력해주세요. (실제 배포시 자동
화 가능)")

    token_input = st.text_input("액세스 토큰 입력 (임시 방식)")
    if st.button("토큰 확인"):
        st.session_state.token = token_input
        st.experimental_rerun()

def chat_interface():
    st.title("🤖 Notion RAG Chatbot")

    # 사이드바
    with st.sidebar:
        st.header("설정")
        if st.button("새 채팅 시작"):
            st.session_state.session_id = str(uuid.uuid4())
            st.session_state.messages = []
            st.experimental_rerun()

    # 채팅 기록 표시
    for msg in st.session_state.messages:
        with st.chat_message(msg["role"]):
            st.markdown(msg["content"])

    # 입력 처리
    if prompt := st.chat_input("질문을 입력하세요..."):
        # 사용자 메시지 표시
        with st.chat_message("user"):
            st.markdown(prompt)
        st.session_state.messages.append({"role": "user", "content": promp
t})

        # API 호출
        with st.spinner("답변 생성 중..."):
            try:
                res = requests.post(
                    f"{API_URL}/api/chat",
                    json={"message": prompt, "session_id": st.session_stat
e.session_id},
                    timeout=60
                )
                if res.status_code == 200:
                    data = res.json()
                    answer = data["response"]
                    sources = data.get("sources", [])

                    full_response = answer
                    if sources:
                        full_response += "\n\n**참고 출처:**\n" + "\n".join
([f"- {s}" for s in sources])

                    with st.chat_message("assistant"):
```

```
                        st.markdown(full_response)
                    st.session_state.messages.append({"role": "assistant",
"content": full_response})
                else:
                    st.error(f"오류 발생: {res.text}")
            except Exception as e:
                st.error(f"연결 오류: {e}")

if not st.session_state.token:
    # 데모를 위해 로그인 패스 가능하도록 수정하거나 로그인 강제
    # 여기서는 로그인 화면 표시
    # login()
    # 편의상 바로 채팅 화면 (실제 사용시 위 login 주석 해제)
    chat_interface()
else:
    chat_interface()
```

## PART 6: 실행 가이드

### 1. Docker 실행

터미널(CMD/Powershell/Terminal)을 열고 프로젝트 폴더로 이동한 뒤 실행합니다.

```
cd notion-rag-agent
docker-compose up -d --build
```

처음 실행 시 이미지를 다운로드하느라 5~10분 정도 소요될 수 있습니다.

### 2. LLM 모델 다운로드

Ollama 컨테이너가 실행된 후 모델을 받아야 합니다.

```
# Llama 3.2 모델 다운로드
docker exec notion-rag-ollama ollama pull llama3.2:3b

# 임베딩 모델 다운로드
docker exec notion-rag-ollama ollama pull nomic-embed-text
```

### 3. 데이터베이스 마이그레이션

MySQL 테이블을 생성합니다.

```
docker exec notion-rag-backend alembic upgrade head
```

### 4. Notion 데이터 수집 (최초 1회)

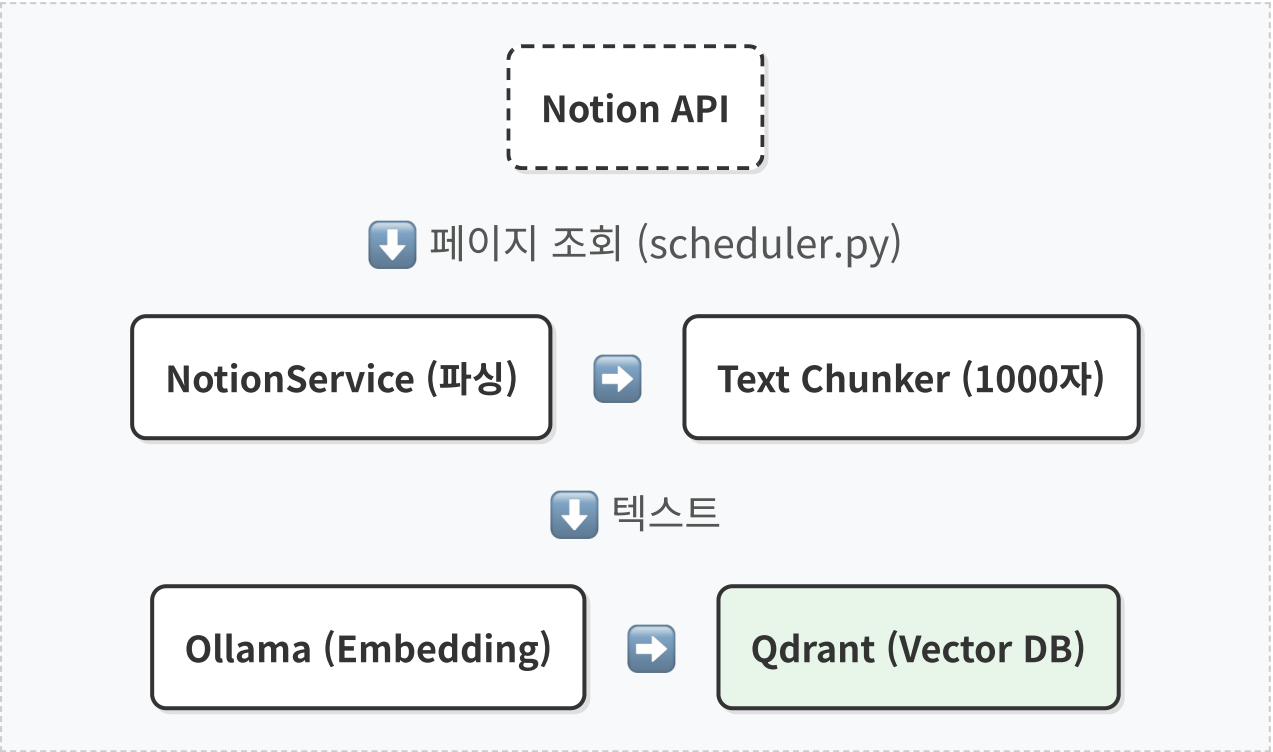
아직 수집 스크립트(ingest.py)를 상세히 작성하지 않았지만, API 엔드포인트를 통해 트리거할 수 있습니다. (추후 구현 필요)

### 5. 접속 확인

- **Frontend:** http://localhost:8501 접속
- **Backend Docs:** http://localhost:8000/docs 접속

## PART 7: 아키텍처 다이어그램 상세

### Notion 데이터 수집 플로우 (Ingestion)




Copyright © 2025 Notion RAG Agent Project. All rights reserved.